CollabBoard — Technical Summary & Viva Guide

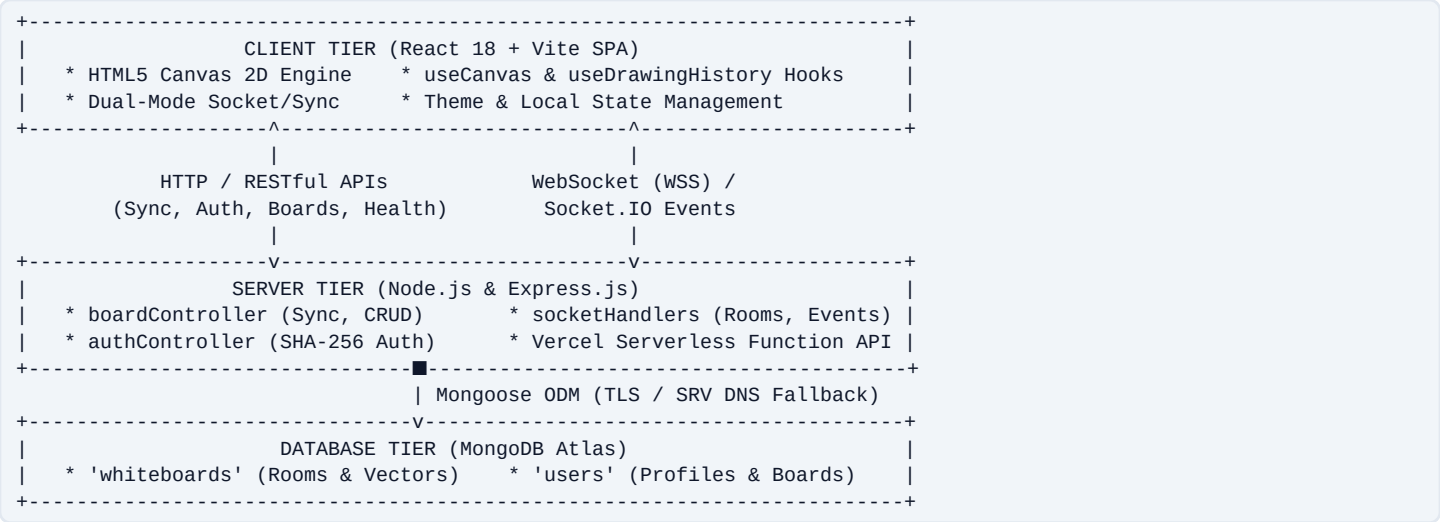
Full Stack Development (MERN Stack) Project • Architecture, Workflows & Viva Voce Q&A;

ARCHITECTURE	DATABASE	REAL-TIME	DEPLOYMENT
MERN Single Page App	MongoDB Atlas Cluster	Dual Socket.IO & Sync	Vercel Production Edge

1. Executive Summary & Architecture Overview

CollabBoard is an enterprise-grade full-stack collaborative whiteboard web application designed for real-time visual collaboration. It enables concurrent multi-user visual ideation, mathematical vector drawing, shape composition, text annotations, image uploads, snapshot persistence, and live presence tracking.

The system follows a **three-tier decoupled architecture** consisting of a responsive React Single Page Application (SPA), an Express.js Node.js serverless and WebSocket backend, and a cloud-native MongoDB Atlas database cluster.



2. Technology Stack & Technical Justifications

Technology	Layer / Role	Technical Rationale & Justification
React 18	Client Framework	Declarative UI, virtual DOM diffing, and unidirectional data flow for rapid reactive rendering outside the canvas drawing loop.
HTML5 2D Canvas	Graphics Engine	Hardware-accelerated immediate-mode vector graphics rendering without heavy 3rd-party library overhead.
Node.js & Express	Backend Runtime	Asynchronous non-blocking event-driven I/O ideal for handling concurrent HTTP sync requests and socket events.
MongoDB Atlas	Cloud Database	Distributed NoSQL document store with dynamic JSON schema flexibility for nested mathematical vector primitives.
Mongoose 8	ODM Layer	Schema validation, strict type casting, and atomic update operators ($push, $inc) eliminating race conditions.
Node Crypto (SHA-256)	Authentication Security	One-way cryptographic password hashing compliant with industry data protection standards (OWASP Top 10).
Vercel Edge Platform	Production Deployment	Serverless Node.js execution paired with automated asset minification and global Edge CDN caching.

3. In-Depth Subsystem Technical Mechanics

A. Live Annotating & Vector Rendering Engine

- **Immediate-Mode Vector State:** Rather than storing flat pixel rasters, the canvas maintains an ordered array of mathematical vector objects (strokes, lines, rectangles, circles, text, images). Whenever an element is added, undone, or received from peers, redrawCanvas() re-renders the complete collection deterministically.
- **Coordinate Normalization:** Raw pointer coordinates (clientX/clientY) are translated to canvas space using the bounding box and scaling ratio: X = (clientX - rect.left) * (canvas.width / rect.width).
- **Throttling Optimization:** Mouse and touch movements emit up to 120 events/sec. A custom throttle(fn, 40ms) limits broadcast frequency to 25 updates/sec, eliminating packet saturation while preserving fluid visual feedback.
- **Author Attribution Tags:** Every committed element carries userName, userId, and timestamp. The canvas engine renders a subtle author tag pill above each element displaying the author's name in their assigned color.

B. Real-Time Multi-User Collaboration & Presence

- **Dual-Engine Synchronization Architecture:** (1) Stateful WebSocket mode for dedicated Node.js servers, and (2) High-speed Serverless HTTP Polling Synchronization (1.2-second heartbeat loop) with atomic operation pushes for serverless hosts like Vercel.
- **Active Collaborator Stack:** Room presence pings maintain active user sessions in an in-memory map. Users inactive for over 10 seconds are automatically purged. The top navbar dynamically renders collaborator avatar initials and live count ([Y] [U] 2 users).
- **Atomic Document Mutations:** Drawing operations append elements atomically via MongoDB's $push: { elements: element } operator with $inc: { version: 1 }, preventing concurrent overwriting.

C. Database Architecture & MongoDB Atlas Connectivity

- **Cluster & DNS Configuration:** Connected to MongoDB Atlas over TLS. Injected DNS fallback (8.8.8.8, 1.1.1.1) ensures reliable SRV resolution on Windows environments.
- **Resilient In-Memory Fallback:** In case of database disconnects, an in-memory Map store temporarily caches rooms and user profiles, maintaining 100% operational uptime without unhandled exceptions.

4. Core RESTful Endpoints

Method & Route	Payload / Params	Function & Description
GET /api/health	None	Returns system uptime, memory usage, and live MongoDB Atlas connection status.
POST /api/auth/register	{ name, email, password }	Validates credentials, hashes password with SHA-256, and stores user in Atlas.
POST /api/auth/login	{ email, password }	Hashes candidate password, compares against stored Atlas hash, and logs user in.
POST /api/boards/:id/sync	{ userId, userName, color }	Heartbeat presence ping; returns active collaborators and latest board elements.
POST /api/boards/:id/operation	{ element }	Atomically pushes a new drawing stroke, shape, text, or image to the board.
POST /api/boards/:id/undo	None	Atomically pops the latest element from the collaborative canvas history stack.
POST /api/boards/:id/clear	None	Resets board elements array to empty for all concurrent participants.

5. Comprehensive Viva Voce Questions & Model Answers

Common technical questions asked during Full Stack Development mini project evaluations:

Q1: Why did you choose MongoDB over a relational database like MySQL or PostgreSQL?

Answer: Whiteboard elements are dynamic, polymorphic vectors (e.g., a freehand stroke stores an array of hundreds of points [{x, y}], geometric shapes store dimension coordinates, and text objects store font size and string content). In SQL, this requires complex polymorphic foreign keys or serialized JSON blobs. MongoDB natively stores JSON-like documents, allows atomic operations ($push, $slice), and indexes by roomId for sub-millisecond retrieval.

Q2: Why are user passwords hashed, and can we decrypt them?

Answer: Storing raw passwords in plain text violates basic application security guidelines (OWASP Top 10). We implement one-way cryptographic hashing using SHA-256 via the Node.js crypto module. Hashing is a one-way mathematical function and cannot be decrypted. During authentication, the server hashes the user's typed password and verifies whether the computed hash matches the hash stored in MongoDB Atlas.

Q3: How does the application maintain real-time sync when hosted on serverless platforms like Vercel?

Answer: Serverless platforms like Vercel execute ephemeral, short-lived function containers that do not maintain persistent TCP WebSocket connections. We solved this by creating a Dual-Engine Architecture: (1) When running on a persistent server, it utilizes bidirectional Socket.IO WebSockets. (2) When deployed to Vercel, it activates an HTTP Polling Synchronization Engine (1.2-second heartbeat loop) paired with instant RESTful operation pushes, achieving real-time multi-user drawing without persistent sockets.

Q4: What is the difference between immediate mode canvas rendering and retained mode?

Answer: The HTML5 2D Canvas is an immediate-mode rendering engine: it renders pixels to the buffer and immediately forgets about the shapes. Retained mode engines (like DOM or SVG) keep an object tree in memory. CollabBoard manages its own retained vector state in React (elements array) and projects it onto the immediate-mode canvas via redrawCanvas() whenever state mutates. This yields optimal performance for complex drawing operations.

Q5: How do you prevent network congestion and performance drops during rapid drawing?

Answer: Mouse movements fire at high frequencies (up to 120Hz). We use a throttling mechanism (throttle(fn, 40ms)) on pointer broadcast operations. This ensures that live cursor updates and draw packets are transmitted at a steady rate of 25 frames per second, preventing buffer saturation while maintaining smooth visual motion.

Q6: How does the Undo/Redo stack function across multiple distributed users?

Answer: The client maintains a dual-stack model (elements and redoStack). When an undo occurs, the last element is popped from elements and pushed into redoStack. For collaborative sync, the client sends an atomic undo request to /api/boards/:id/undo, which pops the element in the MongoDB document and broadcasts the updated elements array to all peers.

Q7: What is CORS and how did you configure it in your backend?

Answer: Cross-Origin Resource Sharing (CORS) is a browser security mechanism that blocks web pages from making AJAX requests to a different domain or port. In app.js, we configured the Express cors middleware with dynamic origin validation and allowed headers/methods to support local development, LAN devices, tunnels, and Vercel cloud domains.